

Redcomm Test for Full Stack Developer (Intern)

Welcome to the Redcomm technical assessment! This practical test is designed to evaluate your full-stack development skills, your architectural decision-making, and your ability to write clean, maintainable code.

Project Overview

You are required to build a simple **Book & Author Management System**. To keep things straightforward and focused on your core development skills, **no authentication system is required**.

The application must allow users to view, create, update, and delete authors and their books, complete with pagination for data lists.

Goals & Evaluation Criteria

Your submission will be reviewed based on the following criteria:

- **Code Quality:** Writing readable, clean, well-structured, and maintainable code that follows industry best practices (e.g., PSR standards for PHP, Vue/Nuxt best practices).
- **Database Design:** Effective utilization of database relationships (e.g., One-to-Many between Authors and Books) and proper migration structures.
- **API Design:** Building logical, consistent, and RESTful API endpoints.
- **State Management & UI UX:** How efficiently data is handled on the frontend and how intuitive the user interface feels.
- **Problem-Solving & Resourcefulness:** Your ability to explain *why* you made certain design decisions and how you leverage modern tools (like AI) responsibly.

Technical Requirements

Backend (API)

- **Framework:** PHP with **Laravel** (Latest stable version preferred).
- **Database:** MySQL / PostgreSQL / SQLite.
- **Features:** * CRUD operations for both Authors and Books.
 - API Pagination for listing endpoints.

Frontend

- **Framework:** **Nuxt.js** (Vue.js).
- **Styling:** **Tailwind CSS**.
- **Features:**
 - A clean interface to manage authors and books.
 - Smooth pagination controls.

Documentation & Transparency

- **README.md:** Clear setup instructions to get the backend and frontend running locally from scratch.
- **EXPLANATION.md:** A short write-up explaining:
 - Why you designed the UI/UX interactions the way you did.
 - Any trade-offs or architectural decisions you made.
 - Any extra optional features you decided to include.
- **AI_USAGE.md (Optional):** If you utilized AI-assisted tools or "vibe coding" (e.g., GitHub Copilot, ChatGPT, Cursor), please include a brief summary or the prompts used. We value resourcefulness—just be transparent about it! Anyway you can use Cursor, Copilot, Trae, Claude Code, or etc.



Submission & Deployed Application (HUGE BONUS)

While a working local setup via your **README.md** is sufficient to pass, **successfully deploying your application to a live, publicly accessible URL will earn you massive bonus points** and instantly set your application apart.

If you choose to deploy, you must provide:

1. **Repository URL:** A link to your public GitHub/GitLab/Bitbucket repository containing the complete source code.
2. **Live Application URL:** A working link to your deployed frontend application (which must communicate correctly with your deployed backend API).



Free Hosting Tips for Students:

You do **not** need to spend any money to deploy this project. We highly recommend utilizing modern free-tier hosting services:

- **For Frontend (Nuxt.js):** Vercel, Netlify, or GitHub Pages.
- **For Backend (Laravel + Database):** Railway, Render, Fly.io, or Supabase.



Bonus Points (Nice to Have)

Want to stand out? Consider implementing one or more of the following:

- **Eager Loading:** Optimizing database queries to prevent the **N+1** query problem when fetching books with their authors.
- **Caching:** Implementing Laravel's cache layer for frequently accessed data (like lists of authors).
- **Advanced UI/UX:** Fully responsive design (mobile-friendly) with smooth loading states or skeletons.
- **Code Documentation:** Docblocks/comments detailing the purpose of key methods.

- **Validation & Error Handling:** Robust backend request validation and user-friendly frontend error alerts.



Tips for Success

Don't overcomplicate it. We value a rock-solid, bug-free core application with great code quality over a massive feature set that is broken or messy. Good luck!